all, the CoffeeScript compiler wraps the code in each compiled file in a scope, essentially doing this:

```
do ->
  # <your program here>
```

Except your CoffeeScript program is compiled to JavaScript, so it's actually wrapped in this:

```
(function() {
  // <your compiled program goes here>
}).call(this);
```

This does the same thing—creates a scope—but it uses the `call` method on the function and passes `this` as the argument. Both `call` and `this` are still relevant to CoffeeScript; if you don't already know them from JavaScript, `this` is covered in chapter 4 and `call` is covered in chapter 6.

3.5.4. Nesting

Functions are first-class values. Not only can you call a function from another function and pass a function to another function, but you can also define a function inside another function where lexical function scope rules still apply. This means that *a function defined inside a function has access to all the variables in the outer function.*

Consider the existing array `flatten` function and how you invoke your `accumulate` function with it:

```
flatten = (acc,current) -> acc.concat current
accumulate([], [[1,3],[2,8]], flatten)
```

Any time you need to flatten an array, you'll need to remember two functions; that's one more than you need. Instead, define a single function that combines them: